

Enhancing Anti-Money Laundering (AML) Detection via Graph Neural Networks

Team: Vincent Nguyen, Vinod Srinivasan, Sarthak Mohindru, Kasyap Bendapudi

Abstract: Money laundering is a financial crime that has significant economic and societal risks, but remains difficult to detect due to its complex transaction patterns. Traditional Anti-Money Laundering (AML) systems rely on rule-based logic and tabular machine learning models, which fail to capture the network structure inherent in financial transactions. In this project, we represent financial transactions as a graph and model fraud detection as a binary edge classification problem. We evaluate the effectiveness of graph-based deep learning approaches by implementing two architectures: a Multi-Principal Neighborhood Aggregation Graph Neural Network (MultiPNA) and a Graph Transformer (FraudGT). Our results show that the multi-PNA GNN achieved the highest F1 score of 0.56 and outperformed our XGBoost benchmark, and that incorporating relational structure through graph neural networks improves the detection of illicit financial activity.

Section 2: Introduction & Motivation

Financial fraud, particularly money laundering, remains a significant challenge for institutions and regulators worldwide. Money laundering involves disguising illicitly obtained funds to make them appear legitimate through complex sequences of transactions across multiple accounts, entities, and countries. This is the “layering” stage of the money laundering cycle (figure 1), where funds are deliberately moved through transaction networks to hide their origin and break the traceable link to the initial illicit activity. By spreading transactions across countries and banks, perpetrators exploit the opacity of the financial systems, making their detection very difficult.

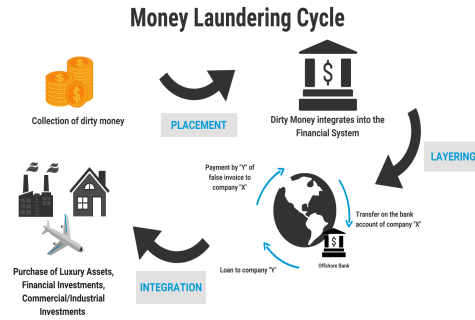


Figure: Money Laundering Cycle

Money laundering poses an economic and societal threat. According to the International Monetary Fund (IMF), illicit financial flows associated with money laundering can distort markets, reduce tax revenues, and undermine economic stability. These activities can weaken financial institutions, fuel inequality, and reduce the inclusiveness and sustainability of economic growth (Claver et al., 2023). Thus, effectively detecting transactions involved in money laundering is essential for all countries and financial institutions.

Historically, financial institutions and regulatory bodies have relied on traditional Anti-Money Laundering (AML) systems governed by rule-based engines. They utilize a static, "if-then" logic, such as automatically flagging any transaction exceeding \$10,000 or flagging a rise in transactions to a high-risk country. While these static rules are effective at catching simple patterns, they fail to capture transactions when the illicit funds are spread across multiple individuals, shell companies, banking institutions, and countries (a process called ‘layering’). Another drawback of the static rule-based approach is that such systems often suffer from high false positive rates and limited adaptability to evolving fraud patterns (Weber et al., 2019).

This project addresses these limitations by leveraging graph-based deep learning models. Financial transaction data can naturally be represented as a graph, where nodes correspond to entities (e.g., accounts or users) and edges represent transactions between them. This representation allows models to capture not only individual transaction features but also the broader structure of interactions within the network. Specifically, we explore the research question: *Can a Graph Neural Network (GNN) outperform traditional machine learning and rule-based systems in identifying illicit transactions by capturing the ‘network’ structure of financial transactions?*

2.1 Task Definition

The primary task in this project is a **binary classification problem**, where the goal is to predict whether a given transaction (i.e. an edge in the graph) is fraudulent or legitimate. We model this as an **edge**

classification task using 2 graph-based neural network architectures: the MultiPNA Graph Neural Network and a Graph Transformer. To benchmark the performance of these models, we compare them against a baseline using XGBoost.

XGBoost serves as a meaningful benchmark because it does not leverage the relational structure of the transaction data and relies solely on our hand-engineered features. So this comparison allows us to evaluate the added value of graph-based modeling and whether using network structure and interactions helps improve money laundering detection over traditional ML-based approaches.

2.2 Evaluation Metrics

Given the highly imbalanced nature of fraud detection tasks and the extreme class imbalance in our dataset (only 0.10% of all transactions in our dataset were fraudulent), accuracy is not a good measure of model performance. Therefore, we use the following evaluation metrics:

1. **F1-Score:** F1-score is the harmonic mean between precision and recall. This helps us address the class imbalance in our data and maintain a balance between correctly identifying actual laundering events (Recall) without flagging thousands of legitimate transactions as fraud (Precision).
2. **Area Under the Precision-Recall Curve (PR-AUC):** Given the severe positive-class imbalance in our data, PR-AUC summarizes the trade-off between precision and recall across different thresholds. We use this over the regular ROC-AUC because PR-AUC focuses on the performance of the rare positive class by evaluating Precision against Recall. So, this makes it a more reliable metric because correctly identifying the rare event while minimizing false positives is the goal.

Section 3: Related Work

Research in fraud detection can be split into two main categories: First is the rule-based and traditional machine learning approaches using hand-engineered features, and the second, and more recent, is using GNNs to capture complex relationships in financial transaction graphs.

The rule-based and traditional ML methods are effective in many settings, but they treat each transaction independently and fail to explicitly model the relational dependencies between entities, limiting their ability to detect network-based fraud patterns. To address these limitations, recent work has explored representing financial transactions as graphs and applying Graph Neural Networks (GNNs) to capture structural information.

Several studies have shown that incorporating graph structure significantly improves fraud detection performance compared to traditional approaches. For example, Asiri et al. applied Graph Convolutional Networks (GCNs) to Bitcoin transaction data and showed that their model outperformed classical methods such as Logistic Regression and Random Forest. Similarly, Lin et al. (2024) proposed two advanced graph-based architectures: Multi-PNA and a graph transformer model called FraudGT, and compared their performance against Random Forest and XGBoost (with graph-derived features). Their results show that graph-based deep learning models outperform traditional machine learning methods, even when the latter had engineered graph features. This highlights both the importance of incorporating relational structure and the superior ability of deep learning models to leverage it effectively. Motivated by these findings, we implement both Multi-PNA and FraudGT in our project.

A newer trend in graph-based fraud detection is the development of specialized architectures tailored to the specific use cases of financial data. Beyond standard GNNs, recent research has introduced hybrid and modified models to better capture temporal and structural patterns. For instance, Verlaan et al. (2025) proposed LAS-GNN, which incorporates an LSTM-based aggregator to model the temporal ordering of transactions. Their research showed strong performance in identifying money laundering motifs by leveraging sequential dependencies, outperforming the previous benchmarks. Similarly, Sui, Mingxiu et al. (2026) proposed a CNN-GNN Fusion Approach that combines Convolutional Neural Networks (CNN) and Graph Neural Networks (GNN) to find suspicious Ethereum transactions. Their model leverages CNNs to pull out local transaction details and GNNs to map how accounts connect. Their hybrid model resulted in 5.79% higher precision and 18.1% higher recall than the previous GNN models.

In summary, prior research has shown a progression from feature-based methods to graph-based learning to modified graph structures for specific use cases such as crypto fraud detection to anti-money laundering. Our work builds on this literature by directly comparing a strong tabular baseline (XGBoost) with two state-of-the-art graph-based approaches of MultiPNA and a Graph Transformer to evaluate the extent to which modeling network structure improves fraud detection performance.

Section 4: Dataset and Exploratory Data Analysis (EDA)

This section explores the dataset's face-value characteristics, initial insights gathered from exploratory analysis, and preprocessing that was performed to help our baseline models generalize better to the data.

4.1. Source

A dataset of 5,078,345 synthetic transactions generated by IBM's AMLSim tool was used for exploratory data analysis and modeling. It consists of 11 features, including the response variable (i.e. 'Is Laundering').

4.2. Exploratory Data Analysis

An initial overview of the dataset reveals a heavy class imbalance within the response variable, in which only 0.1019% of all observations are illicit transactions. This equates to 5,177 transactions. The class imbalance must be addressed when modeling the data in order to obtain better generalization results.

proportion	
Is Laundering	
0	0.998981
1	0.001019

In regard to numerical variables, the distribution of the transaction amount is right-skewed for both benign and illicit transactions, though the effect is much greater for benign transactions. Transaction amounts tend to be much lower for benign transactions, the median amount being 1407.51 as opposed to 8667.21 for illicit transactions.

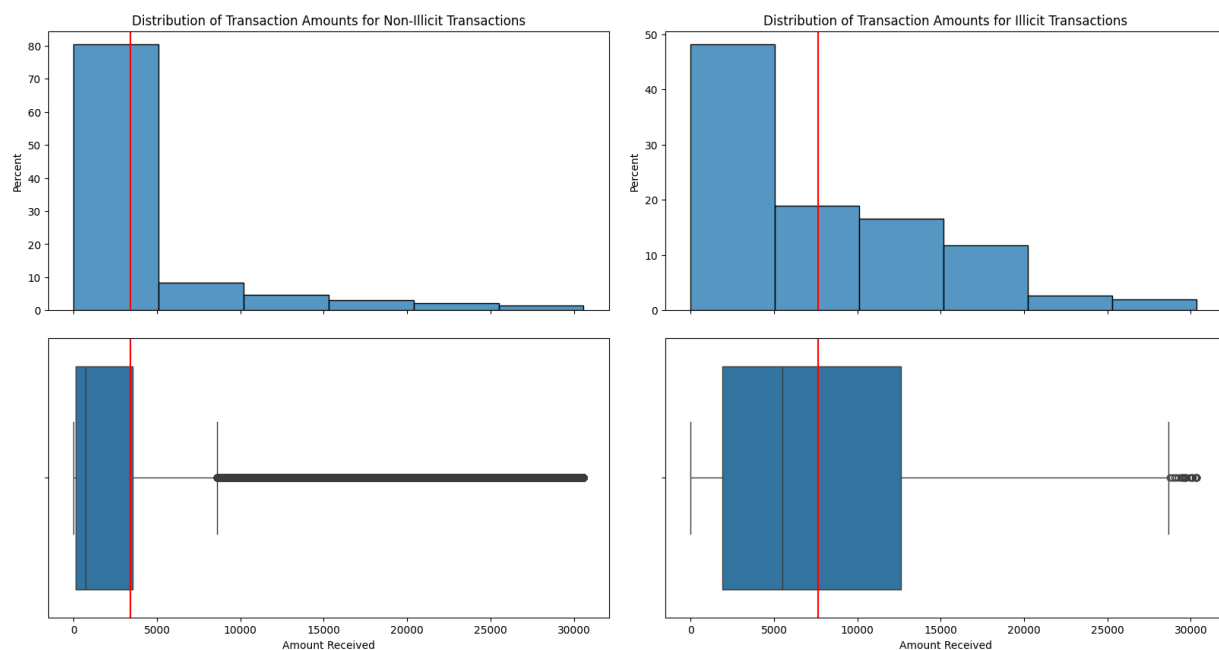


Figure: Transaction Amounts by category

When comparing the distribution of currencies by payment format, a few specific payment formats stick out between benign and illicit transactions. For benign transactions, cheque and credit card payments make up at least 60% of payments across the top 10 most recurring currencies. On the flip side, ACH is the most frequent payment format by a landslide for illicit transactions, especially for the Saudi Riyal currency (i.e. 96.52% of transactions under that currency).

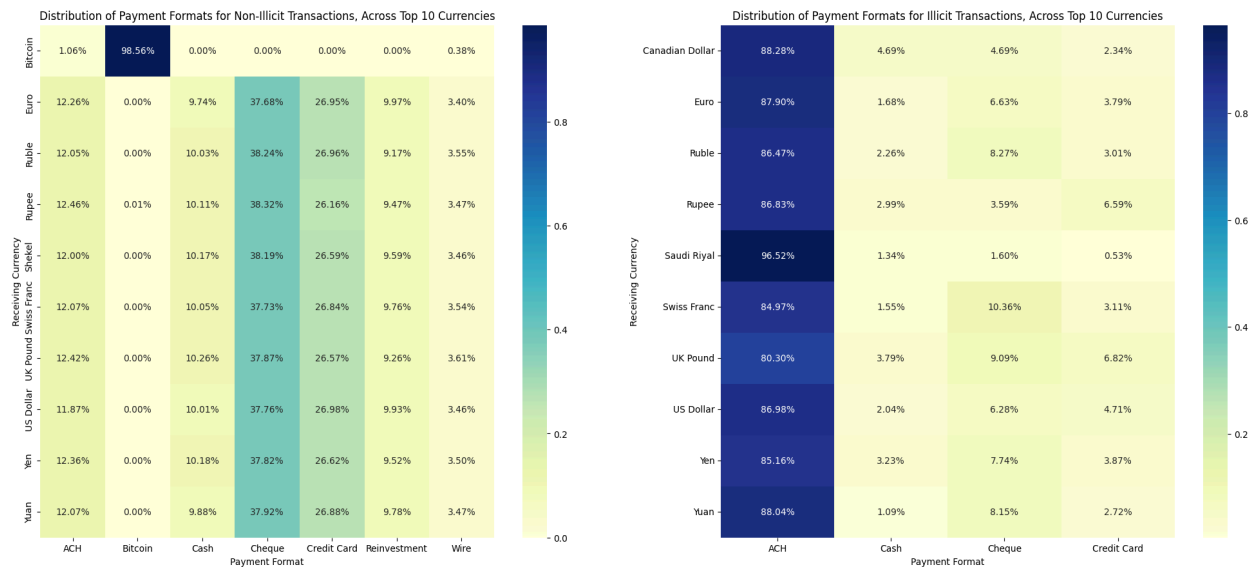


Figure: Heatmap by currency for transaction type

The U.S. dollar has a slightly above a 35% share of transactions in the dataset regardless of whether it's benign or illicit, but this level of consistency isn't the same for other countries' currencies. The Saudi Riyal wasn't one of the top 10 most frequent currencies among benign transactions, but it's the third most frequent currency among illicit transactions, taking 7.22% share.

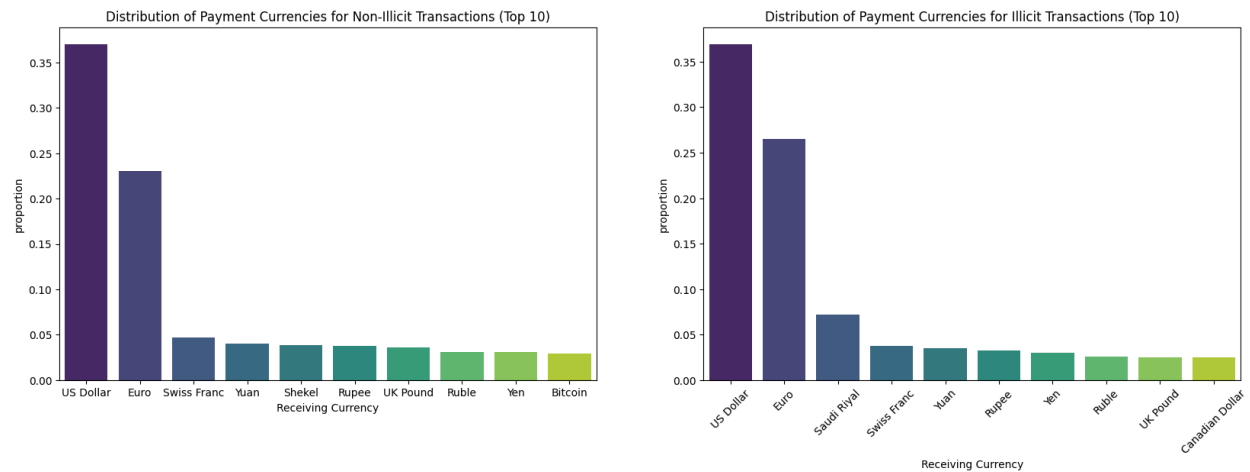


Figure: Bar graph by currency for transaction type

4.3. Preprocessing

Because the original dataset is tabular, it had to be recontextualized into a graph format in order for the two GNN baselines to train properly upon the observations. Hence, feature engineering and data augmentation was performed with the idea of a graph structure in mind. Data pertaining to account information was to be contained within nodes, while transaction details aside from the former were to be embedded in the edges connecting the nodes.

4.3.1. Feature Engineering

The NetworkX library was used to create a directed graph between the sender (i.e. the 'Account' column) and receiver (i.e. the 'Account.1' column).

From there, the centrality metrics computed during the creation of NetworkX's directed graph were used to craft the following node features for each unique account:

- **'PageRank'**: the overall importance of an account to the graph as a whole
- **'Hub_Score'**: the measure of an account's importance as a sender, weighted by the importance of the accounts it sends money to
- **'Authority_Score'**: the measure of an account's importance as a receiver, weighted by the importance of the accounts sending money to it
- **'In_Degree' and 'Out_Degree'**: the number of in-going edges and out-going edges, respectively

On top of NetworkX's graph features, the following features were added to the list of node features:

- **'Unique_Recipients' and 'Fan_In'**: the number of accounts receiving money from the node, along with the number of accounts sending money to the node respectively
- **'Avg_Amount_Recv'**: the average amount received by an account per transaction

Regarding edge features, velocity and deviation features were crafted for each transaction:

- **'Time_Since_Last_Txn'**: the time elapsed between the current transaction and the previous transaction the sender was involved in
- **'Count_1h' and 'Count_24h'**: the rolling number of transactions involving the sender account as the sender, over the past hour and over the past 24 hours respectively
- **'Sum_1h' and 'Sum_24h'**: the rolling amount of money the sender account sent over the past hour and over the past 24 hours respectively
- **'Amount_vs_Avg_Ratio'**: the ratio between the amount received in the transaction and the mean amount of money the sender sends

4.3.2. Data Augmentation

After feature engineering was performed, each of the node features and edge features, along with the **'Amount Received'** and **'Amount Paid'** columns from the original dataset, were scaled to a (0, 1) value range through the use of `MinMaxScaler()`. The normalization was done with the intent of stabilizing gradients during model training.

As for categorical features belonging to the original dataset, the following list of features, [**'Receiving Currency'**, **'Payment Currency'**, **'Payment Format'**, **'From Bank'**, **'To Bank'**], went through label encoding such that each category in a feature was assigned a unique integer to identify it. Afterwards, those features were scaled to a (0, 1) value range as well. They, along with **'Amount Received'** and **'Amount Paid'**, were edge features.

4.3.3. Graph Construction

In addition to the aforementioned node features and edge features, it was necessary to obtain a tensor of edge indices and edge labels. Hence, a 2D tensor of each edge’s connecting nodes and a 1D tensor of each transaction’s class (i.e. Not Laundering vs. Laundering) were crafted.

PyTorch Geometric’s `Data()` function was then used to construct the graph-structured dataset containing node features, edge features, edge indices, and edge labels.

4.3.4. Train/Validation/Test Split, Undersampling, and LinkNeighborLoader

A random 70/10/20 train/val/test split was performed on the graph-structured dataset, though stratified sampling was used to make sure the original class imbalance was maintained across all 3 subsets before undersampling was performed.

The training data was then randomly undersampled such that there was a 20:1 ratio between benign transactions and illicit transactions.

Afterwards, `LinkNeighborLoader()` was used to shuffle and create batches of subgraphs consisting of 4096 observations each for the training, validation, and test sets, which not only helps with model training but also prevents an out-of-memory error due to an overly large graph size. Each of these subgraphs are based around the notion of sampling the neighbors of specific “seed” edges, capturing local dependencies for edge classification. In the case of the training set, a modification was made to its `LinkNeighborLoader()` function: only edges belonging to illicit transactions were sampled, such that the models will rely heavily on those edges during training.

Section 5: Methodology

5.1 Multi-Principal Neighborhood Aggregation Graph Neural Network (MultiPNA GNN)

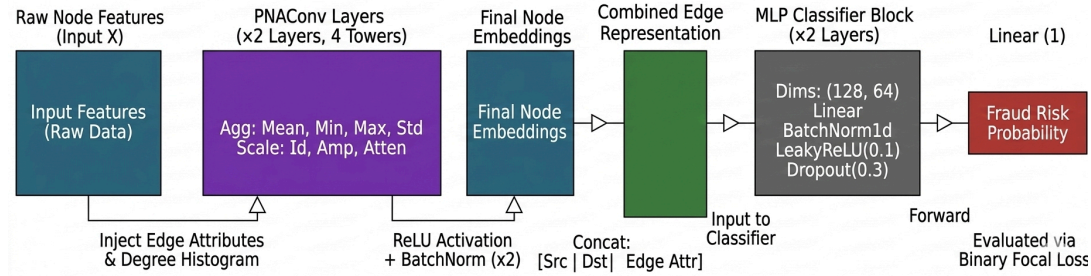


Figure: Multi-PNA Model Overview

Model overview: Input node features are first passed through two PNAConv layers, producing the final node embeddings. For each edge, the corresponding source node embedding, destination node embedding, and edge attributes are then concatenated and fed into an MLP classification head, which outputs a binary prediction after applying the loss function.

The key distinction between models like XGBoost with graph-derived features, a standard GCN, and the MultiPNA model lies in the PNAConv layer. This layer performs multiple aggregation and scaling operations, enabling it to capture richer structural information from the graph. This layer performs a grand total of 12 operations, hence the name. Specifically, these operations fall into two categories: **aggregation and scaling**.

A standard GCN relies solely on mean aggregation, but a PNAConv layer does 4 different kinds of aggregation operations: Mean, Min, Max, and standard deviation. Each of these captures a different aspect of node neighborhood behavior, and the usefulness of each of these operations is defined as follows:

- 1. Standard Deviation:** Captures variability in transaction patterns. Suppose a user performs ninety-nine \$1 transactions and a one \$10,000 transaction. The mean is \$101, which seems like a normal amount, but the standard deviation would be \$995, which would be flagged by the system for anomalous behavior.
- 2. Max:** In the same example, the \$10,000 transaction would be immediately identifiable through the max aggregator.
- 3. Min:** Sometimes hackers input extremely small amounts like \$0.01 or \$0.5 to check if the transactions are going through or not. These are flagged when min is used as a criterion.
- 4. Mean:** Standard mean aggregation that is also used in GCNs.

Thus, by having these extra aggregation options, the system is a lot more robust to various fraudulent transactions and types.

In addition to multiple aggregation functions, the PNAConv layer also incorporates three types of **scaling mechanisms**, which adjust messages based on node degree:

1. Identity Scaling: No special use, just ensures the message passes as is.

2. Amplification Scaling: This ensures that nodes that are more densely connected have amplified signals, and sparsely connected nodes are not.

$$S_{amp} = \frac{\log(d+1)}{\log(\delta+1)}$$

3. Attenuation Scaling: This dampens the signal of massively connected nodes so they don't dominate the network, while slightly boosting the signal of sparsely connected nodes so they don't get ignored.

$$S_{att} = \frac{\log(\delta+1)}{\log(d+1)}$$

Overall, the combination of 4 aggregation functions and 3 scaling mechanisms results in 12 distinct operations, which give the MultiPNA model its expressive power. This flexibility allows the model to capture a wide range of transactional behaviors, making it well-suited for anti-money laundering tasks, as shown in the results section.

5.2 Graph Transformer (FraudGT)

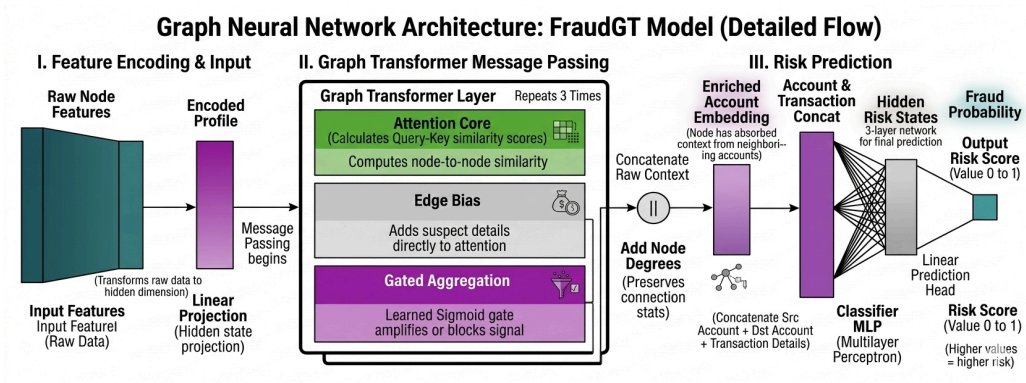


Figure: Graph Transformer Model Overview

Model overview: The flow is as seen in the figure, raw node features are projected into a linear space for consistency, and passed into a specialized Graph Transformer layer called the FraudGT layer, found in

literature [4]. This layer is applied iteratively three times, the first time for immediate neighbours, the second for neighbors of immediate neighbors and the third for neighbors that are 3 spaces away.

Following this, information about the in-degree and out-degree of each node is added. These embeddings (Source Node, Destination Node and Edge attributes) are concatenated and passed onto the MLP classification head, to finally give us a binary output after the BinaryFocalLoss function.

To improve model expressiveness, there is a sigmoid gating mechanism prior to the final classification layer. This gate learns to adaptively weight the combined embeddings, allowing the model to selectively amplify more informative features while blocking less relevant ones. This helps the model focus on important structural and transactional signals. We ran an experiment where we included and dropped the sigmoid gate from the model, and we found that the inclusion of the gate gave us better performance.

Reasons for the results of FraudGT being sub-par compared to the MultiPNA:

1. Softmax: FraudGT calculates attention scores (alpha) and applies a softmax. Softmax forces all the attention weights of a node's neighbors to sum up to exactly 1.0. This means the Transformer is ultimately calculating a weighted average of the neighborhood. Averages alone are not good enough for catching fraud, which is why the MultiPNA wins out on this point.

2. Blindness to graph scale: Financial graphs follow a strict Power Law, a few merchant nodes have millions of connections, while most normal users have very few.

Even though FraudGT uses attention to weigh which neighbor is most important, it does not understand the scale of the node itself compared to the rest of the graph. A node with 5,000 transactions and a node with 2 transactions are treated with the same mathematical perspective. MultiPNA avoids this by the use of scaling as discussed previously, hence winning out.

3. Parameter count: FraudGT has 50,000 parameters, whereas MultiPNA has 147,000 parameters. With 141 node features and 13 edge features, FraudGT simply lacks the capacity to map out all the complex non-linear interactions compared to the much wider MultiPNA model.

Section 6: Experimental Setup

The following section details the hardware, software, compute times, and hyperparameters utilized during our experiments to ensure the replicability of our 2 deep learning models: Multi-Principal Neighborhood Aggregation Graph Neural Network (*MultiPNA GNN*) and Fraud Graph Transformer (*FraudGT*).

6.1. Hardware and Software

All models were trained on a Windows-based PC equipped with a GPU to accelerate computation. Both models were implemented in Python using PyTorch and PyTorch Geometric (PyG).

The full hardware and software configuration is as follows:

1. **GPU:** NVIDIA RTX 4060 (8GB VRAM)
2. **CPU:** Ryzen 7 8845 HS (32 GB RAM)
3. **Framework:** PyTorch (v2.10.0) & PyG (v2.7)
4. **Programming Language:** Python (v3.10)
5. **Compute Environment:** CUDA-enabled GPU training

6.2 Compute Times

We report both the average wall-clock time per training epoch and the total training time for each model.

MultiPNA:

- Average time per epoch: **~57 seconds**
- Total training time: **~173 minutes** (~2.9 hours)

FraudGT:

- Average time per epoch: **~24.8 seconds**
- Total training time: **~39 minutes** (~0.64 hours)

The MultiPNA model required substantially more computation per epoch and overall training time compared to FraudGT. This is likely due to more intensive neighborhood aggregation and message-passing operations. Additionally, MultiPNA has approximately $3\times$ as many trainable parameters as FraudGT (147k vs. 50k), further increasing computational cost.

Total Compute Usage:

Accounting for final training runs (~3.5 hours), failed runs, hyperparameter tuning, experimentation, and graph construction/feature engineering, we estimate the total compute usage for this project to be approximately **18–20 GPU hours**.

6.3 Hyperparameters

We selected hyperparameters through tuning based on validation performance & prevailing literature..

MultiPNA:

- **Batch size: 4096** via LinkNeighborLoader (using 25 and 15 neighbors for 1st and 2nd hop sampling, respectively). This ensures we don't get OOM errors.
- **Optimizer:** Adam
- **Initial learning rate:** 0.001
- **Learning rate scheduler:** ReduceLROnPlateau
 - Mode: max
 - Factor: 0.5
 - Patience: 3
 - Interpretation: ReduceLROnPlateau reduced the learning rate by a factor of 0.5 after 3 epochs without improvement in the validation metric.
- **Dropout Rates:** 0.3 (applied within the downstream MLP edge classifier)
- **Weight decay:** 1e-4
- **Network Parameters:** 64 hidden dimensions with 4 PNA towers.

FraudGT:

- **Batch size: 2048** via LinkNeighborLoader (using 25 and 15 neighbors for 1st and 2nd hop sampling, respectively). This ensures we don't get OOM errors.
- **Optimizer:** Adam
- **Initial learning rate:** 0.001
- **Learning rate scheduler:** ReduceLROnPlateau
 - Mode: max
 - Factor: 0.5
 - Patience: 3
 - Interpretation: ReduceLROnPlateau reduced the learning rate by a factor of 0.5 after 3 epochs without improvement in the validation metric.
- **Weight decay:** 1e-5
- **Dropout rate:**
 - 0.2 within FraudGTConv message passing layers
 - 0.3 in the final MLP edge classifier
- **Network Parameters:** 64 hidden dimensions, 8 attention heads, and 2 overall message-passing layers.

Table: Summary of hyperparameters

<u>Hyperparameter(Row) / Model (Column)</u>	<u>MultiPNA GNN</u>	<u>FraudGT</u>
Batch Size	4096	2048
Optimizer	Adam	Adam
Learning Rate	0.001	0.001
LR Scheduler	ReduceLROnPlateau	ReduceLROnPlateau
Weight Decay	1e-4	1e-5
Dropout (Message Passing)	N/A	0.2
Dropout (MLP Classifier)	0.3	0.3

Section 7: Quantitative Results

Table: Table of model results

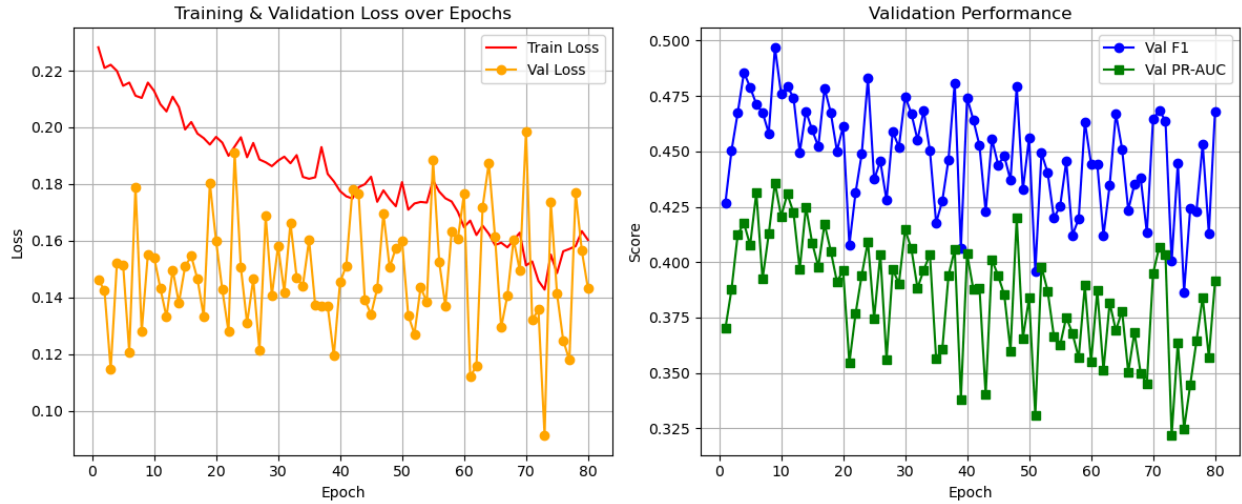
Metric (Row) / Model (Col)	XGBoost	MultiPNA GNN (Best Run)	FraudGT (without Sigmoid Gate)	FraudGT (with Sigmoid Gate)
<u>F1 Score</u>	0.48	0.56	0.26	0.32
<u>Precision</u>	0.59	0.73	0.27	0.50
<u>Recall</u>	0.41	0.45	0.24	0.24
<u>PR AUC</u>	0.45	0.47	0.15	0.38

Between each of the baseline models, the MultiPNA graph neural network performed the best by a sizable margin in regards to the F1 score, the primary evaluation metric measured for the problem at hand.

It should be noted, however, that an F1 score of 0.48 and a PR AUC of 0.41 was achieved on the latest run, the results of which will be displayed on the loss curves and plots in the following subsections; the better results from the previous run can be attributed to randomness in the training process (e.g. seeds, random initializations).

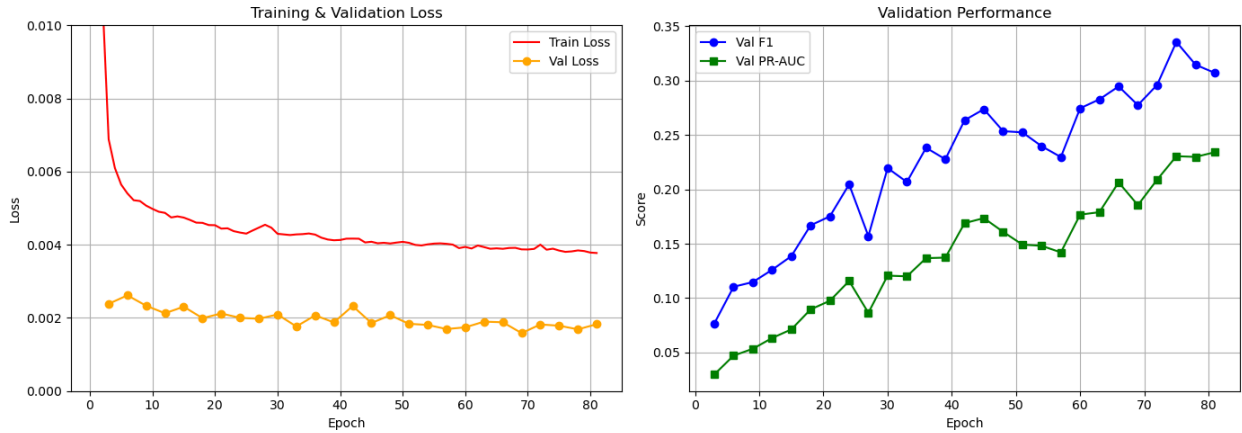
7.1. Learning Curves

7.1.1. MultiPNA GNN



While the training loss consistently decreased throughout the 80 epochs, the same couldn't be said for the validation loss and the F1 score for the validation loss. Since the validation loss oscillated so often throughout the 80 epochs and didn't show any genuine improvement, it can be concluded that the model either overfit the data or the learning rate was too high.

7.1.2. FraudGT (without Sigmoid Gate)



In regards to the training loss, the training loss curve for the FraudGT model was smoother compared to that of the MultiPNA GNN, not to mention it converges earlier. In addition, its loss is much lower on paper. However, it encounters a similar problem as the MultiPNA GNN model: the validation loss doesn't really improve throughout the 80 epochs. This means the model failed to properly learn from the data.

Going back to the discussion regarding the FraudGT's loss being lower than that of the MultiPNA model, the metric can be deceptive, given that raw accuracy isn't the primary evaluation metric but rather the F1 score due to the heavy imbalance between class labels.

7.2. Class-wise Metrics

7.2.1. XGBoost

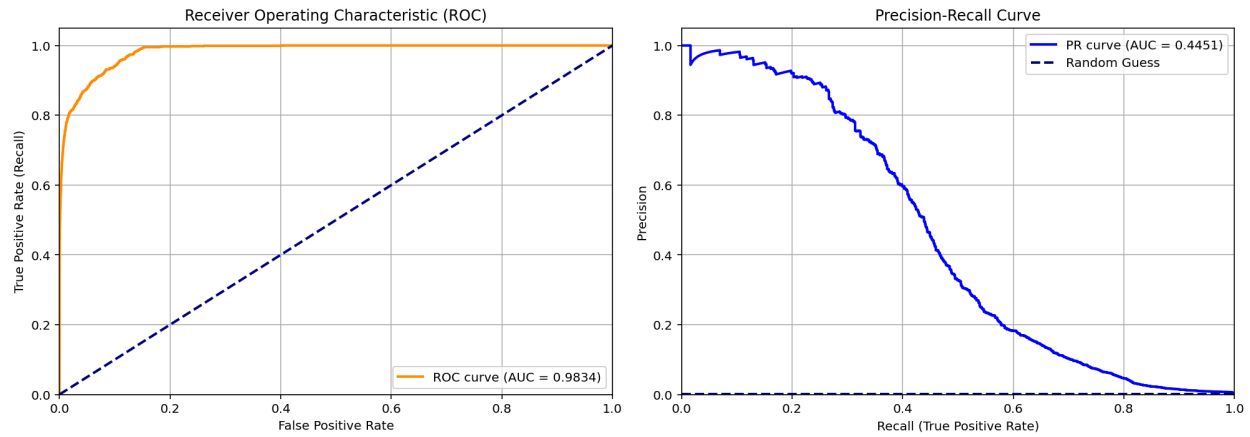


Figure: ROC-AUC & PR-AUC curve

While an AUC of 0.50 typically indicates that the model results are no better than random guessing, the fact that only 0.10% of all observations are illicit transactions indicates that the PR-AUC of 0.4451 for the XGBoost is actually pretty promising and can even be considered to be “good”.

7.2.2. MultiPNA GNN: Total Wall-Clock Time - 172.73 minutes
Average Time per Epoch - 57.37 seconds

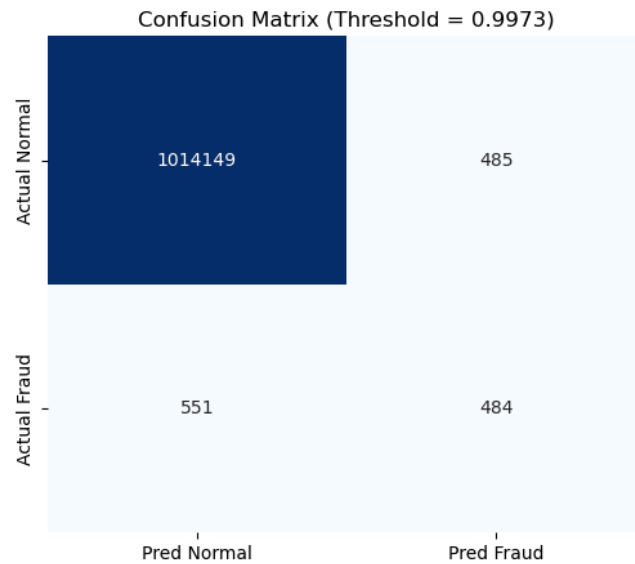


Figure: Confusion Matrix

The confusion matrix for the MultiPNA GNN’s class predictions shows a particular story: benign transactions were orders of magnitude more common than illicit transactions, so predictions for benign transactions were much more accurate. On the other hand, among the actual fraudulent transactions in the test set, only 46.76% of them were correctly predicted.

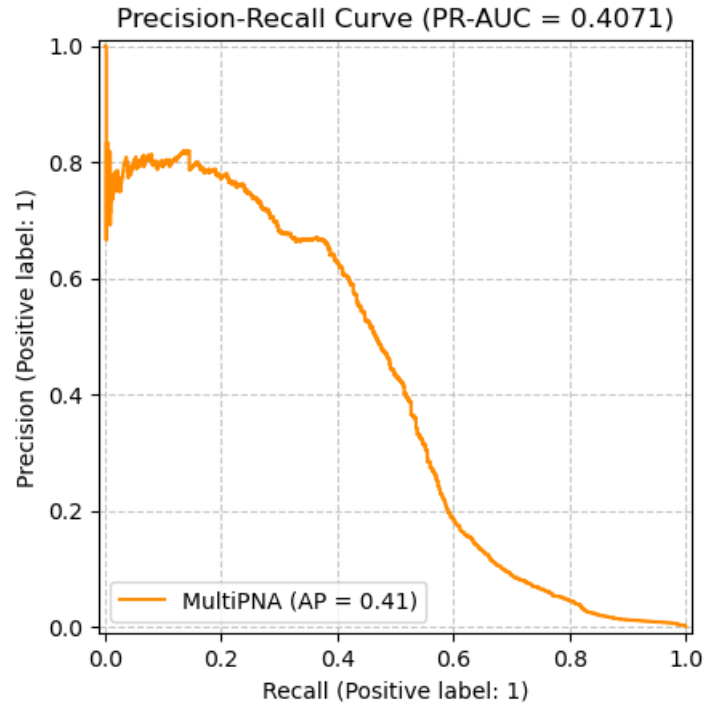


Figure: PR-AUC curve

As discussed with the case of the XGBoost model, a PR-AUC of 0.41 for the MultiPNA would be considered a promising result in the case of a heavily imbalanced dataset such as the IBM synthetic transactions dataset.

7.2.3. FraudGT (without Sigmoid Gate): Total Wall-Clock Time - 38.60 minutes
Average Time per Epoch - 24.82 seconds

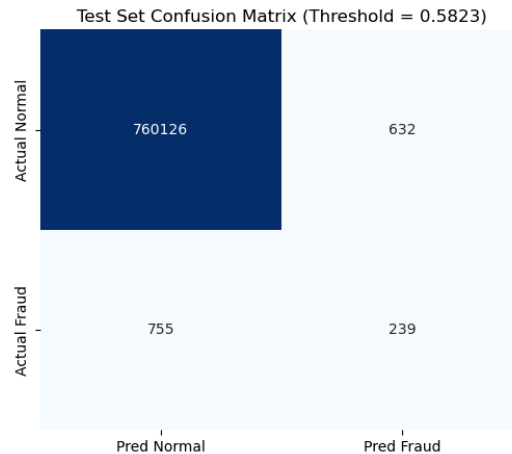


Figure: confusion matrix

Although the FraudGT model had a much lower loss than the MultiPNA did, the former did a lot worse at predicting which transactions were actually illicit, among all the actual illicit transactions in the test set; the correct prediction rate among that class is 24.04%, as opposed to 46.76% from the MultiPNA model.

PR-AUC curve:

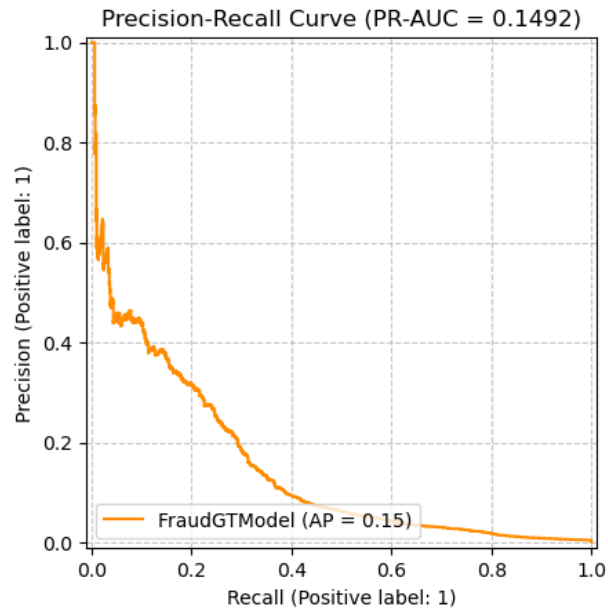


Figure: PR-AUC curve

Compared to the PR-AUC of the XGBoost and MultiPNA models, the PR-AUC for the FraudGT (without Sigmoid gate) heavily underperformed. The density of 0.1492 is a lot closer to the notion that the result is equivalent to random guessing, in spite of the heavy class imbalance giving some leeway.

Section 8: Discussion, Ablations, & Error Analysis

8.1 Discussion of Results

Our experimental results confirm that GNNs provide a significant advantage over traditional tabular baselines like XGBoost for anti-money laundering tasks. The MultiPNA architecture was the strongest performing model, achieving an F1 score of 0.56 compared to XGBoost’s 0.48. The success of the MultiPNA model is driven by its use of multiple aggregators; specifically, the max, min and standard deviation aggregators are more effective at capturing outlier signals than the standard averaging approach used in basic attention mechanisms.

In contrast, the FraudGT underperformed with an F1 of 0.32. We attribute this partly to the inherent difficulty of applying self-attention to an extremely imbalanced dataset where only 0.80% of transactions are illicit. Softmax can unintentionally dampen rare fraudulent signals when they are surrounded by a massive volume of benign transactions, making it harder for the model to distinguish illicit motifs.

To stabilize these results, we implemented a Learning Rate Scheduler. Given the extreme 0.80% class imbalance, a static learning rate proved too aggressive, often leading to unstable convergence. By using a scheduler, the model automatically reduced its step size when the validation loss plateaued, allowing for more precise convergence and helping the GNN layers effectively navigate the complex loss surface.

8.2 Ablation Study: Sigmoid Gate Results

We noticed that our transformer model performed worse than the XGBoost benchmark and significantly below our PNA GNN. These were unexpected results, as we expected the FraudGT to beat the XGBoost benchmark at a minimum. This led us to hypothesize that the sigmoid gate following the attention layers in the transformer might be blocking important signals from being passed on in the model. As a result, we decided to test the transformer model without the sigmoid gate.

We conducted this ablation study to measure the specific impact of the gate on the FraudGT architecture. Our testing showed that removing the gate caused the F1 score to drop further, from 0.32 to 0.26. This confirms that the Sigmoid Gate is a necessary component for noise reduction in this dataset. It functions as a critical filter that helps the model focus on suspicious connections while ignoring the "noise" created by the 99.2% of legitimate transactions.

8.3 Error Analysis

The model demonstrates strong performance in general classification, though detection sensitivity remains a key area for optimization in high-stakes environments like fraud detection. Rather than a failure, we observed a strategic bias toward the majority class; this leads to False Negatives in "low-connectivity" nodes where a lack of neighbor data makes structural flagging difficult. Additionally, the model produces False Positives for legitimate high-volume "hubs" that structurally mimic criminal "mixers." These results are promising, proving that GNNs can isolate complex patterns that traditional models miss, even within a highly skewed data distribution.

Section 9: Conclusion and Future Work

9.1. Conclusion

Although the performance of the MultiPNA model exceeded that of the XGBoost baseline model and was good for the extreme amount of class imbalance in the data, it's hard to find the results to be fully satisfactory because it ultimately isn't that effective at predicting which transactions are illicit, from the lens of the F1 score. Additionally, the Transformer model performed a lot worse than the MultiPNA model, which was unexpected from our end; additional work would have to be done to fully break down what exactly caused that model in particular to perform so badly relative to even the XGBoost baseline. However, we can conclude that the PNA model had superior aggregation methods due to tackling the message passing process with several summary statistics that could provide different viewpoints of a node's neighbors. Hence, the problem was only partially solved from our end.

During the modeling process, we ran into numerous limitations. For one, due to the extremely large size of the dataset, it was imperative to find ways to pass the data along to visualization methods or the 2 GNN models such that out-of-memory errors wouldn't pop up. Secondly, issues pertaining to the installation of PyTorch Geometric libraries were introduced when trying to pass the LinkNeighborLoader() to the model training process, across both Jupyter Notebook and Google Colab. Lastly, the training process was extremely slow, which limited our ability to run experiments on other parameters or architecture configurations.

9.2. Future Work

Given a hypothetical six month extension and a tenfold increase in computational resources, we would expand the study in several ways:

- Explore different ways of addressing the class imbalance between the majority and minority class. We were particularly concerned about the “shattered graph problem”, in which downsampling our tabular dataset would result in edges and nodes getting deleted. This poses an issue because not only could important sender or receiver accounts get deleted entirely, but we intended for our model to capture money laundering patterns such as “layering”, the movement of money through several intermediate accounts to throw off suspicion. Hence, simply randomly deleting the “legitimate-looking” nodes from existence could backfire on us heavily. Additional care should be put into this.
- A future research direction could be to explore hybrid architectures that integrate PNA convolutional layers with transformer-style attention mechanisms. This would allow the model to benefit from the rich, multi-aggregator neighborhood representations of PNA while also leveraging attention scores to prioritize important node and edge interactions during embedding updates
- Given a tenfold increase in computational power, we can afford to not only up the complexity of the model but also engage in heavy hyperparameter tuning through grid search and other methodologies. One drawback of our data and model was that training was pretty slow due to how computationally heavy everything was. Since we based our GNN models off of papers, we could definitely take a look at other papers and come to our own conclusions about what would make a GNN model work on the IBM synthetic transactions dataset in particular.

If we were to adapt this project into a Masters thesis, one avenue of change we could look towards is using synthetic data generation in particular to address the issue of class imbalance, as discussed previously. In that case, we'd be able to create new observations for the minority class without having to worry about deleting dependencies between existing nodes. Ultimately, expanding the scope of our

project to that of a Master's thesis would involve having to make several new considerations such that our study adds something fresh to the topic of AML detection.

References:

1. Claver, C., El Khoury, C., & Weeks-Brown, R. (2023). Financial crimes hurt economies and must be better understood and curbed. International Monetary Fund (IMF).
2. Weber, M., Chen, J., Suzumura, T., Pareja, A., Ma, T., Kanezashi, H., Kaler, T., Leiserson, C. E., & Chen, T. (2019). Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics. arXiv preprint arXiv:1908.02591.
3. Asiri, A., Somasundaram, K. Graph convolution network for fraud detection in bitcoin transactions. Sci Rep 15, 11076 (2025). <https://doi.org/10.1038/s41598-025-95672-w>
4. Lin, J., Guo, X., Zhu, Y., Mitchell, S., Altman, E., & Shun, J. (2024). FraudGT: A simple, effective, and efficient graph transformer for financial fraud detection. In Proceedings of the 5th ACM International Conference on AI in Finance (ICAIF '24) (pp. 292–300). ACM. <https://doi.org/10.1145/3677052.3698648>
5. Stan Verlaan, Ioana Hulpuş, and Erik Jan van Leeuwen. 2025. LAS-GNN: A Graph Neural Network for Temporal Money Laundering Motif Detection. Proceedings of the 6th ACM International Conference on AI in Finance. Association for Computing Machinery, New York, NY, USA, 256–264. <https://doi.org/10.1145/3768292.3770410>
6. Sui, Mingxiu and Su, Yiyun and Shen, Jiaqing and Zhang, Wenchao, Intelligent Anti-Money Laundering on Cryptocurrency: A CNN-GNN Fusion Approach (January 02, 2026). Available at SSRN: <https://ssrn.com/abstract=6180382> or <http://dx.doi.org/10.2139/ssrn.6180382>
7. GitHub link: <https://github.com/Sarthak0047/STATS-426-Final-Project>